CloudForge AI - Project Report

Project: CloudForge AI - AI-Assisted Cloud Infrastructure and DevOps Control Plane
Repository: https://github.com/asvittal17/CloudForge
Branch at submission: `audit/antbox-submission`
Report date: October 2026

1. Abstract

CloudForge is a web platform that turns a plain-language infrastructure request into a reviewed, scanned, and gated Terraform deployment. A user describes what they need ("a Postgres database and a Python API service"), a locally hosted LLM produces an architecture plan, the platform translates it into Terraform, and the resulting artifact travels through a strict pipeline: **stage -> validate -> security-scan -> plan -> approve -> apply -> deployed**. Every state-changing step is authenticated and ownership-checked; real deployments are guarded by an explicit kill switch, saved-plan integrity digests, an atomic status claim, and a fail-closed security scanner. The result is a working demonstration that AI-generated infrastructure can be made *auditable and safe to review* rather than blindly executed.

2. Problem Statement

AI code generation makes producing infrastructure-as-code fast, but it also makes shipping unreviewed, insecure configurations dangerously easy. The problem this project addresses:

- 1. **Trust gap.** LLM output must not become live cloud resources without structural validation (terraform fmt/validate), security scanning (Checkov), and a human approval that is bound to the *exact* artifact that will run.
- 2. **Blast-radius control.** A demo or audit environment must not be able to create billable AWS resources by accident. CloudForge enforces this at multiple layers, defaulting to fail-closed.
- 3. **Plan reuse risk.** A classic Terraform `/tmp/plan` race is that the plan a human approves is not the plan that gets applied. CloudForge closes it with SHA-256 digests recorded at plan time and re-verified immediately before apply.

3. Requirements and Scope

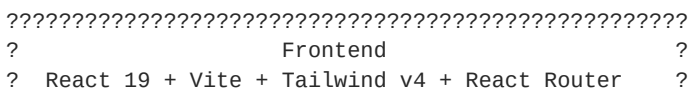
In scope

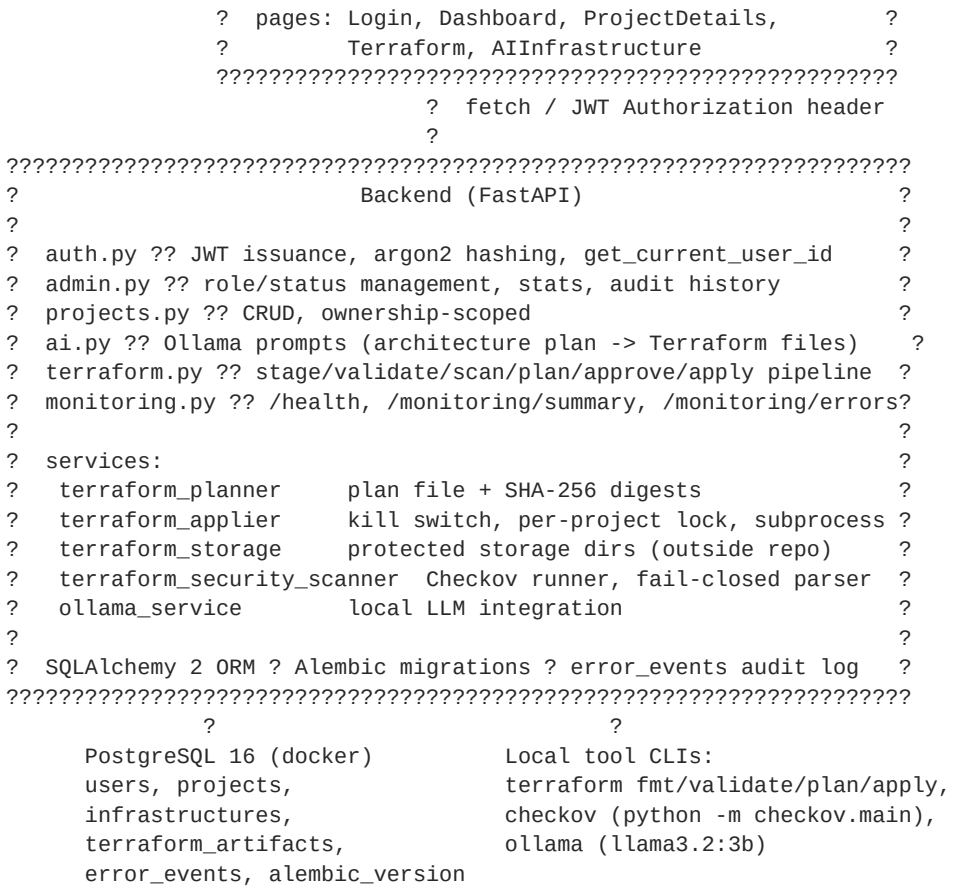
- Multi-user platform with registration, login (JWT), roles (`user`/`admin`), account enable/disable, and self-service projects.
- AI generation of an architecture plan and Terraform configuration via local Ollama.
- Terraform artifact lifecycle with Checkov scanning, saved plans, approval, and gated apply.
- Monitoring: real component health, per-category error/audit event log, platform statistics.
- CI pipeline for backend tests, dependency checks, frontend lint/build, and `npm audit`.

Out of scope (documented limitations, not hidden)

- Remote Terraform backends (S3 + DynamoDB locking) - the exact production prerequisite.
- Multi-node apply serialization (the in-process lock is single-node).
- Cloud credential management / actual AWS account wiring.

4. Architecture





Protected Terraform storage (`CLOUDFORGE_TERRAFORM_STORAGE`, default `D:\CloudForge-data\terraform`) lives **outside the repository and outside any web root**, holding saved plan files, plan-time configuration snapshots, and local development state, per project/artifact.

5. Technology Stack and Justification

Choice | Justification

FastAPI | Typed, async-capable, first-class Pydantic validation; the routers remain thin because services own the logic.
SQLAlchemy 2 + PostgreSQL | Relational integrity between users -> projects -> infrastructures -> artifacts; JSON columns where flexible data (AI plan) is needed.
Alembic | Versioned migrations; the initial schema is a *real* migration (`be2717ee7e9b`), so a fresh database rebuilds the exact production schema.
argon2 + python-jose (JWT) | Modern password hashing; short-lived signed access tokens.
React 19 + Vite + Tailwind v4 | Fast dev loop, small tooling footprint, utility CSS keeps bundle CSS lean.
Ollama (llama3.2:3b, local) | No cloud AI dependency, no data leaves the machine, deterministic offline demo.
Terraform CLI (subprocess) | Real plan/applied semantics without an SDK abstraction that could drift from real behavior.
Checkov | Widely used IaC scans; invoked as a Python module (`python -m checkov.main`) to avoid fragile `.cmd` shims on Windows.
pytest, GitHub Actions | Standard regression + CI story for the assignment.

6. Application and Infrastructure Workflow

- 1. **Describe** - user enters a natural-language request on the AI page; Ollama returns a JSON architecture plan.
- 2. **Generate** - the plan is converted to `main.tf`, `variables.tf`, `outputs.tf` and staged as a `TerraformArtifact` (status `staged`).

- 3. **Validate** - `terraform fmt` + `terraform validate` run against the staged files; formatting diffs and errors are returned verbatim.
- 4. **Scan** - Checkov runs fail-closed: scanner crash or malformed report ? scan fails ? pipeline stops. Findings (passed/failed counts and failing checks) are surfaced to the UI.
- 5. **Plan** - `terraform plan -out=approved.tfplan` writes the saved plan into protected storage; SHA-256 digests of the plan file and the configuration snapshot are recorded on the artifact row (`plan_sha256`, `config_sha256`, `planned_at`); status becomes `plan_ready`.
- 6. **Approve** - re-loads the saved plan file, verifies its digest still matches, records `approved_at`, and emits a `terraform_approval` audit event.
- 7. **Apply** - re-verifies digests, performs an atomic conditional `UPDATE ... WHERE status='approved'` claim to `applying` (a concurrent second request gets 409), re-checks the kill switch *inside the applier immediately before the subprocess*, applies the **saved plan file** (no new plan), and transitions to `deployed` - or honestly back to `approved` on failure, with the error preserved and a `terraform_apply` audit event.

The kill switch `CLOUDFORGE_ALLOW_TERRAFORM_APPLY` defaults to disabled. While disabled, the apply endpoint returns an honest HTTP 403 explaining that real applies are disabled; nothing is launched. CI forces it to `0`.

7. Database Design

Table | Purpose | Notable columns

`users` | Accounts and roles | email (unique), hashed_password (argon2), role, is_active
`projects` | Ownership boundary for everything else | name, description, owner_id -> users.id
`infrastructures` | Last-known desired architecture per project | project_id (unique), architecture_plan (JSON)
`terraform_artifacts` | The deployable unit | project_id, main_tf, variables_tf, outputs_tf, status, plan_sha256, config_sha256, planned_at, approved_at
`error_events` | Audit/monitoring trail | source, category, message, severity, route, http_status, created_at

Migrations: `be2717ee7e9b` (initial schema) then `0f76c165a878` (saved-plan integrity columns). Verified on a disposable database: a fresh cluster plus `alembic upgrade head` reproduces the live schema exactly and ends at head `0f76c165a878`.

8. Security and Approval Model

Authentication & authorization

- Argon2 password hashing; JWT bearer tokens with expiry.
- Every project-scoped route resolves the project through a single preflight (`_get_owned_project`) that filters on `project.id AND owner_id`; another user's project is indistinguishable from a missing one (404) - this blocks insecure direct object references.
- Admin routes require `role = "admin"`; admins cannot demote themselves; account disable/enforce is a separate explicit action.

Pipeline safety (defense in depth)

- 1. Authenticated, ownership-checked route (404 on foreign projects).
- 2. State-machine guards: plan only from `staged`, approve only from `plan_ready` (after digest verification), apply only from `approved`.
- 3. Saved-plan integrity: digests verified after approval and again inside the applier immediately before the subprocess; a missing or altered plan is refused at both points.
- 4. Atomic DB claim `approved -> applying` prevents duplicate concurrent applies (409 for the loser).

- 5. Kill switch checked at the route *and* re-checked inside the applier immediately before any `terraform apply` subprocess could launch.
- 6. Per-project threading lock serializes applies; the lock is a context manager, so even a crash inside the critical section cannot wedge it (regression-tested).
- 7. Crash recovery: on startup, artifacts stranded in `applying` from a killed process are reset to `approved` so they can never appear deployed when they are not.
- 8. Failed applies return status to `approved` (retryable) and record an error audit event; success records an info event. Nothing pretends success.
- 9. Secrets: the RDS password is passed only as `TF_VAR_database_password` in the subprocess environment, never written to disk or returned to the client; `.env` files and saved plans are git-ignored and stored outside the repo.

Honesty controls

- `/health` reports real component probes (database, Ollama) as healthy/degraded/unhealthy - never fabricated data.
- The scanner report parser tolerates compact Checkov output but fails closed when structure is unrecognizable.

9. Testing Methodology and Actual Results

Backend - `pytest`: 74 passed (0 failed, 0 skipped in this environment).

Coverage includes: the full artifact state machine (invalid transitions rejected), saved-plan round-trip with the real Terraform CLI (plan creation, `terraform show` readback - clearly separated classes; the real-CLI tests skip automatically when Terraform is absent and never run `apply`), digest-tampering and missing-plan refusals at both the router level and *inside the applier*, kill-switch behavior at the route and immediately before the subprocess, atomic claim semantics, lock release after an exception, the closed-loop failure path (failed apply returns to `approved` with the real error), ownership 404 preflight, audit-event emission on approval and failed apply, admin authorization, rate-limited frontend error ingest (429 verifies), and the live Checkov scanner against real findings plus compact/malformed-report regression tests.

Backend - `pip check`: no broken requirements. (A cosmetic `~ip` invalid-distribution warning remains; see §11.)

Frontend - actual results: `npm ci` success; `npm audit` 0 vulnerabilities; `npm run lint` exit 0; `npm run build` exit 0. Vite reports one warning: the main JS chunk is ~512.7 kB, slightly above Vite's 500 kB advisory threshold (see §11).

Live end-to-end (manual, against a running backend): cross-tenant access returns 404; a failed plan leaves the artifact at `staged` with null digests; a successful plan records matching digests and the plan file loads via `terraform show`; approval produces `approved` + `approved_at`; apply with the switch disabled returns 403 and the status is untouched; a real Checkov scan reported 24 passed / 2 failed checks with genuine findings.

CI: `.github/workflows/ci.yml` (name `CI`, least-privilege `permissions: contents: read`) runs backend pytest + `pip check` and frontend `npm ci`/`npm audit`/lint/build with `CLOUDFORGE_ALLOW_TERRAFORM_APPLY: "0"`. The YAML validates locally. The workflow file is new and untracked at report time, so a real GitHub Actions run could not yet be observed - it will run once the branch is pushed.

No real infrastructure was created, and no destructive command was ever executed, during this project's audit.

10. Screenshots

No screenshots are included because none were captured in a reproducible way during this audit. The demo script (§12) describes what each screen shows so a reviewer can reproduce the visuals by running the demo.

11. Limitations and Future Work

- 1. **Local Terraform state.** Development mode uses `-backend=false` with state inside protected storage. There is no distributed lock or state versioning. *Production prerequisite:* a remote backend (e.g., S3 + DynamoDB) - configuration only; no code change is implied.
- 2. **Single-node serialization.** The per-project apply lock is in-process plus a DB status guard; multiple backend nodes would still be DB-guarded, but a remote backend's native locking is the robust answer.
- 3. **Bundle size.** The main JS chunk (~512.7 kB) is marginally above Vite's advisory 500 kB. Route-level lazy loading (`React.lazy`) is the non-blocking optimization; it was deliberately not bundled into a security-focused change set.
- 4. **`~ip` warning from pip.** A leftover invalid distribution from an interrupted pip self-upgrade inside the development virtualenv. pip itself ignores it; removing the `~ip` / `~ip-24.0.dist-info` folders from `backend\.venv\Lib\site-packages` (or recreating the venv) clears the warning. Cosmetic; no dependency is broken (`pip check` is clean).
- 5. **CI run not yet observed on GitHub** because the workflow has not been pushed at report time.
- 6. Ollama quality is bounded by the local 3B model; prompts are structured to constrain output but hallucinated HCL is always possible - which is precisely why the deterministic validate/scan/plan gates exist downstream.

12. Setup and Demonstration

Setup: see the repository README (Quick Start: docker compose, venv, `.env`, `alembic upgrade head`, uvicorn, `npm run dev`). Demo: `docs/demo-script.md` (5-7 minutes, no AWS required; fallback path included). Likely interview questions with code-grounded answers: `docs/interview-prep.md`.